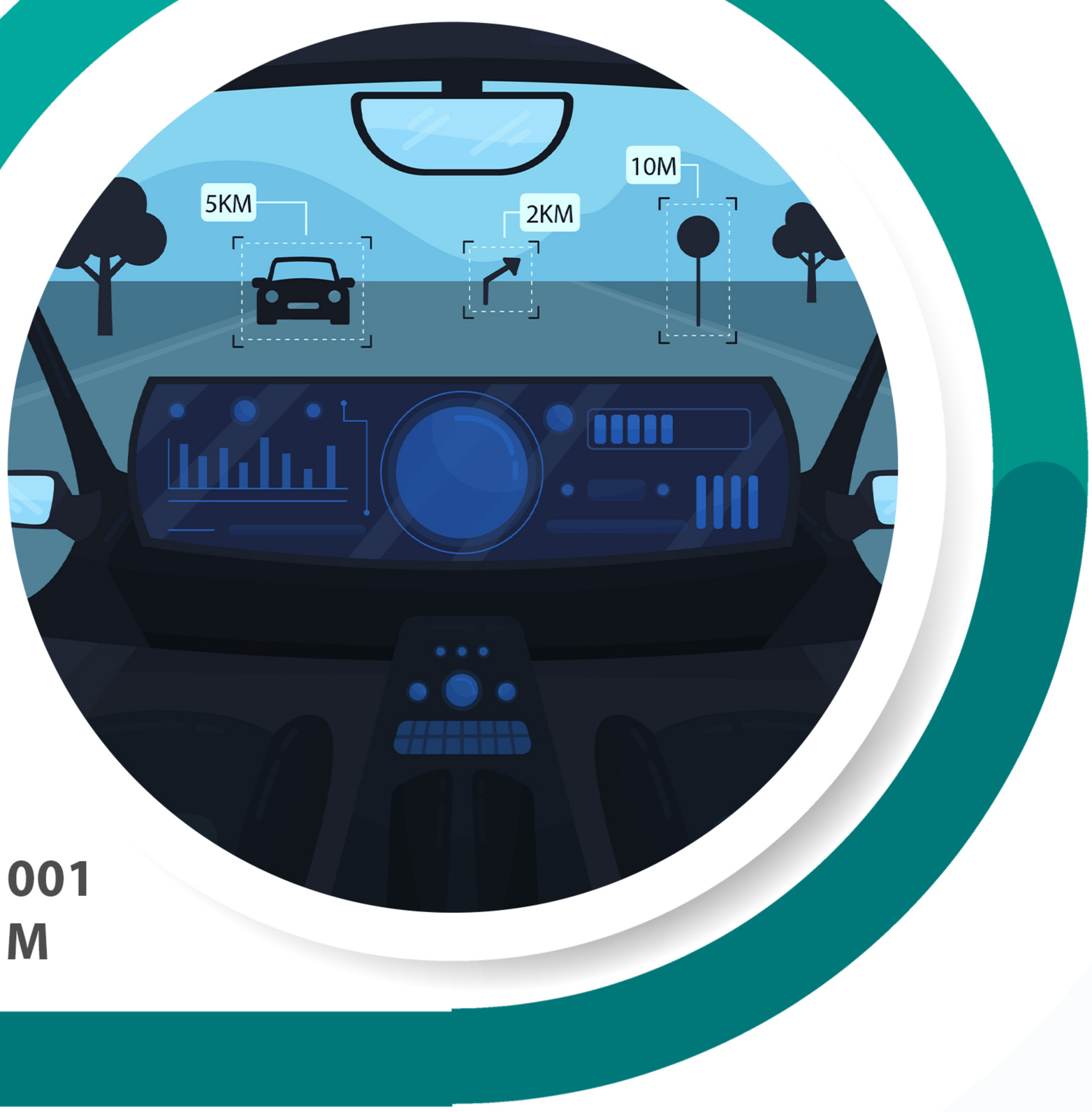


Vehicle Detection

Team ID: SDCC2021001

Presenter: Hareesh M

PROJECT - 5 REPORT

Submitted by: Team Avida of *National Institute of Technology Agartala*

For: SAE Self Driving Car Challenge 2022

Vehicle Detection

PROJECT REPORT

For SAE India (Southern Section) SDCC

Submitted by,

Team AVIDA (NIT Agartala)

in partial fulfillment for the completion of

SAE Self-Driving Car Challenge

Under the guidance of,

Mr. Shanmugam Sivaraman

(Project Mentor)

DECLARATION

I the undersigned solemnly declare that the project report titled LANE FINDING BASIC is based on my own work carried out during the course of our study under the supervision of Mr. Shanmugam Sivaraman, Secretary, SAE India Southern section.

I assert the statements made and conclusions drawn are an outcome of my research work. I further certify that,

I. The work contained in the report is original and has been done by me under the general supervision of my supervisor.
II. The work has not been submitted to any other institution for any other degree/diploma/certificate in this university or any other university of India or abroad.
III. I have followed the guidelines provided by the university in writing the report.
IV. Whenever I have used materials (data, theoretical analysis and text) from other sources, I have given due credit to them in the references.

KeerthanaNeigapula
Hareesh M
Shivansh Mishra
Dikshita Jaraut
Sowjanya Matakala
Sumit Anand
Anawil Tiwari
Krushnakant Borude
Thanesh Bisen
Rakshit Gupta

Date – 09/06/2022

ACKNOWLEDGEMENTS

I would like to use this opportunity to thank my mentor Mr. Shanmugam Sivaraman, Secretary, SAE India Southern section. for guiding me throughout the project duration. I would also like to thank SAE India Southern Section for providing me an opportunity to work on this interesting project.

I owe gratitude to various books and online resources for the completion of this project. These resources have been mentioned in the references.

CONTENTS

S No.	Title
	Cover
	Declaration
	Acknowledgements
1	Objective
2	Introduction
3	Package & Libraries
4	Methodology
5	Code
6	Result Discussion
7	Learnings & Conclusion
	References

Project # 5

Vehicle Detection:

Objective:

The goal of the project was to develop a pipeline to reliably detect cars given a video from a roof-mounted camera.
In this project, we will perform Real-time vehicle detection in a video or from camera streams using OpenCV and a pre-trained cascade model.

Introduction:

Real-time vehicle detection is one of the many application of object detection, whereby focuses on detecting cars within an image together with the location coordinates.

Vehicle detection together with road detection is heavily applied on self-driving cars, for a car to navigate safely along the road, it has to know where other cars are positioned so as it can avoid a collision.

In addition, vehicle detection is being used in traffic surveillance systems in such a way it can detect the traffic based on number vehicle and use that data to manage and control it. Computer vision is one of the hottest topics in Artificial Intelligence. It is being used extensively in Robotics, Autonomous Vehicles, Image Classification, Object Detection & tracking, Semantic Segmentation as well as in various photo correction apps. In Autonomous vehicles, vision remains the main source of information to detect lanes, traffic lights, and other visual features.

Our objective is to capture the coordinates of the moving object and highlight that object in the video. Consider this frame from a video below:

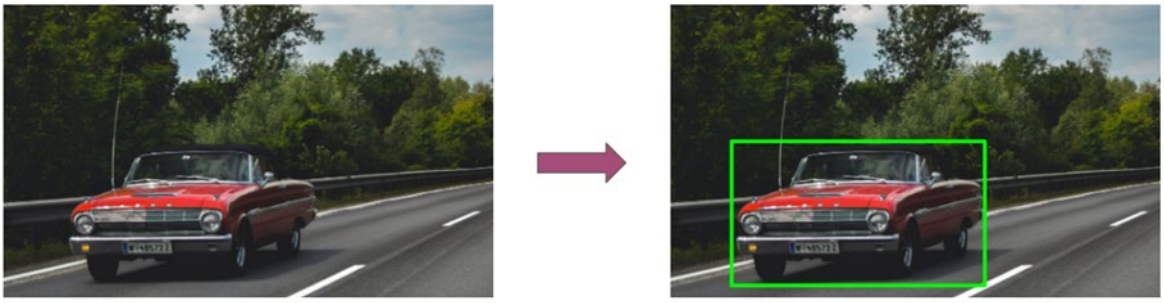


Fig: Vehicle Detection

We would want our model to detect the moving object in a video as illustrated in the image above. The moving car is detected and a bounding box is created surrounding the car.
There are multiple techniques to solve this problem. You can train a deep learning model for object detection or you can pick a pre-trained model and fine-tune it on your data. In our approach, we shall use a pre-trained HAAR Cascade Classifier file. This will help us in not only saving time but also getting more accurate results as the pre-trained data will already consist of a lot of pre-existing data that it can work on to make a good decision.

Packages and libraries:

We at first need to make sure we have all the software packages installed.

1. OpenCV

OpenCV stands for "Open Source Computer Vision" is a library for computer vision and machine learning software library invented by Intel in 1999. OpenCV has C++, Python, Java and MATLAB interfaces and supports Windows, Linux, Android, and Mac OS. OpenCV has been written natively in C++ and has a templated interface that works seamlessly with STL containers.

a. You can install OpenCV using *pip package installer*.pip install opencv-python. The detailed installation steps can be found here as well.
b. After the installation, try importing the package to make sure, if everything is working fine.

```
import cv2
cv2.__version__
```

If everything works perfectly fine, without any errors, you are good to go!

2. NumPy

NumPy is the fundamental package for scientific computing in Python. It is a Python library that provides a multidimensional array object, various derived objects (such as masked arrays and matrices), and an assortment of routines for fast operations on arrays, including mathematical, logical, shape manipulation, sorting, selecting, I/O, discrete Fourier transforms, basic linear algebra, basic statistical operations, random simulation and much more.

a. NumPy can be installed with conda, with pip, with a package manager on macOS and Linux, or from source
b. If you use pip, you can install NumPy with:
   pip install numpy
c. Also when using pip, it's good practice to use a virtual environment - see Reproducible Installs below for why, and this guide for details on using virtual environments.
d. If your installation fails with the message below, see Troubleshooting ImportError.

Methodology:

1. Computer Vision

Computer vision is a field of artificial intelligence (AI) that enables computers and systems to derive meaningful information from digital images, videos and other visual inputs — and take actions or make recommendations based on that information. If AI enables computers to think, computer vision enables them to see, observe and understand.

Computer vision works much the same as human vision, except humans have a head start. Human sight has the advantage of lifetimes of context to train how to tell objects apart, how far away they are, whether they are moving and whether there is something wrong in an image.

Computer vision trains machines to perform these functions, but it has to do it in much less time with cameras, data and algorithms rather than

retinas, optic nerves and a visual cortex. Because a system trained to inspect products or watch a production asset can analyse thousands of products or processes a minute, noticing imperceptible defects or issues, it can quickly surpass human capabilities.

2. Thresholding

In digital image processing, thresholding is the simplest method of segmenting images. From a grayscale image, thresholding can be used to create binary images. The simplest thresholding methods replace each pixel in an image with a black pixel if the image intensity is less than a fixed value called the threshold, or a white pixel if the pixel intensity is greater than that threshold. In the example image on the right, this results in the dark tree becoming completely black, and the bright snow becoming completely white. Below is before and after thresholding an image.

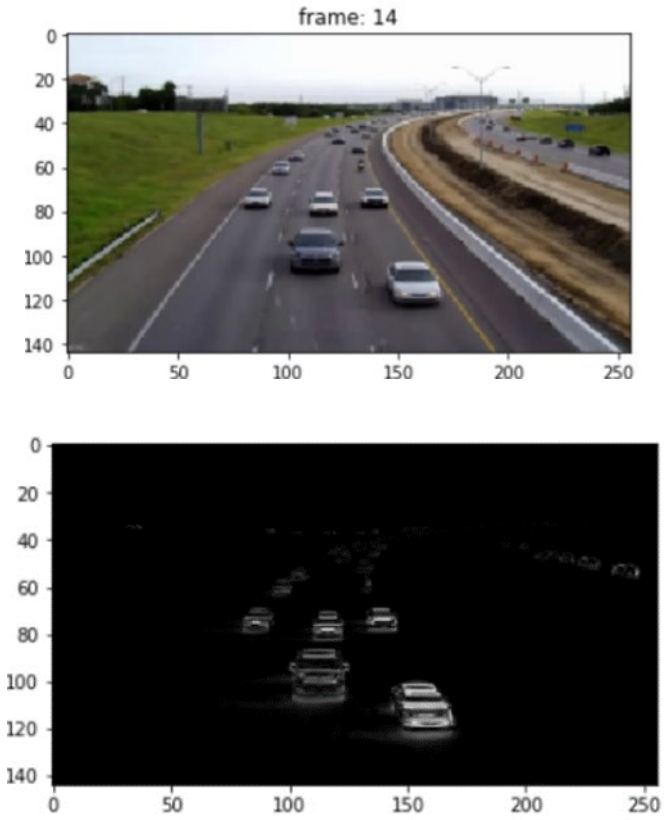


Fig: Thresholding

It is done in 3 steps:-

a. Grayscale image specify the image to be processed, in our case a grayscale image is going to be image fetched from the video streams.
b. ScaleFactor specify how much the image size is reduced at each image scale, a good value is mostly chosen as 1.05
c. minNeighbors specify how many neighbours each candidate rectangle should have, to retain it. This parameter will affect the quality of the detected faces

3. Cascade Classifier

Object Detection using Haar feature-based cascade classifiers is an effective object detection method proposed by Paul Viola and Michael Jones in their paper, "Rapid Object Detection using a Boosted Cascade of Simple Features" in 2001. It is a machine learning based approach where a cascade function is trained from a lot of positive and negative images. It is then used to detect objects in other images.

Here we will take example of a face detection model using cascade classifier. Initially, the algorithm needs a lot of positive images (images of faces) and negative images (images without faces) to train the classifier. Then we need to extract features from it. For this, Haar features shown in the below image are used. They are just like our convolutional kernel. Each feature is a single value obtained by subtracting sum of pixels under the white rectangle from sum of pixels under the black rectangle.

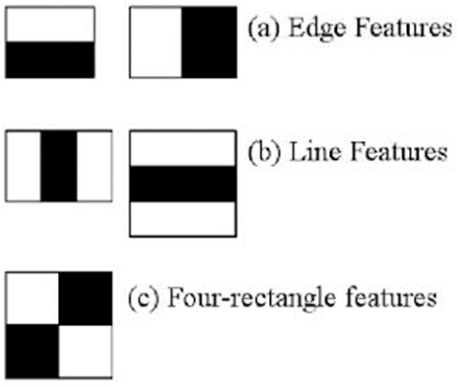


Fig: Cascade Classification

Now, all possible sizes and locations of each kernel are used to calculate many features. (Just imagine how much computation it needs? Even a 24x24 window results over 160000 features). For each feature calculation, we need

to find the sum of the pixels under white and black rectangles. To solve this, they introduced the integral image. However large your image, it reduces the calculations for a given pixel to an operation involving just four pixels. Nice, isn't it? It makes things super-fast.

However, among all features we calculated, most of them are irrelevant. For example, consider the image below. The top row shows two good features. The first feature selected seems to focus on the property that the region of the eyes is often darker than the region of the nose and cheeks. The second feature selected relies on the property that the eyes are darker than the bridge of the nose. However, the same windows applied to cheeks or any other place is irrelevant. So how do we select the best features out of 160000+ features? Adaboost achieves it.

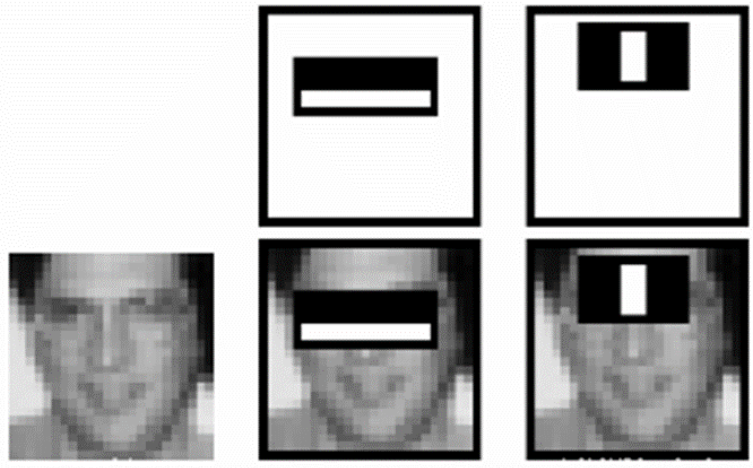


Fig: Training Image

For this, we apply each feature on all the training images. For each feature, it finds the best threshold, which will classify the faces to positive and negative. Obviously, there will be errors or misclassifications. We select the features with minimum error rate, which means they are the features that most accurately classify the face and non-face images. (The process is not as simple as this. Each image is given an equal weight in the beginning. After each classification, weights of misclassified images are increased. Then the same process is done. New error rates are calculated. Also new weights. The process is continued until the required accuracy, error rate is achieved, or the required number of features are found).

The final classifier is a weighted sum of these weak classifiers. It is called weak because it alone cannot classify the image, but together with others

forms a strong classifier. The paper says even 200 features provide detection with 95% accuracy. Their final setup had around 6000 features. (Imagine a reduction from 160000+ features to 6000 features. That is a big gain).

So now, you take an image. Take each 24x24 window. Apply 6000 features to it. Check if it is face or not. Wow! Isn't it a little inefficient and time consuming? Yes, it is. The authors have a good solution for that.

In an image, most of the image is non-face region. Therefore, it is a better idea to have a simple method to check if a window is not a face region. If it is not, discard it in a single shot, and do not process it again. Instead, focus on regions where there can be a face. This way, we spend more time checking possible face regions.

For this they introduced the concept of Cascade of Classifiers. Instead of applying all 6000 features on a window, the features are grouped into different stages of classifiers and applied one-by-one. (Normally the first few stages will contain very many fewer features). If a window fails the first stage, discard it. We do not consider the remaining features on it. If it passes, apply the second stage of features and continue the process. The window, which passes all stages, is a face region. How is that plan!

The authors' detector had 6000+ features with 38 stages with 1, 10, 25, 25 and 50 features in the first five stages. (The two features in the above image are actually obtained as the best two features from Adaboost). According to the authors, on average 10 features out of 6000+ are evaluated per sub-window.

Therefore, this is a simple intuitive example of how Cascade Classifier for face detection works. Similar to it we can use cascading techniques on other objects such as traffic on road and various cars, which is how we have achieved the objective of this project. We have used a car cascade classifier that can be found here.

These trained cascade classifiers are usually being stored in the XML format, therefore you should download the cascade that was trained to detect cars and have it in the project directory.

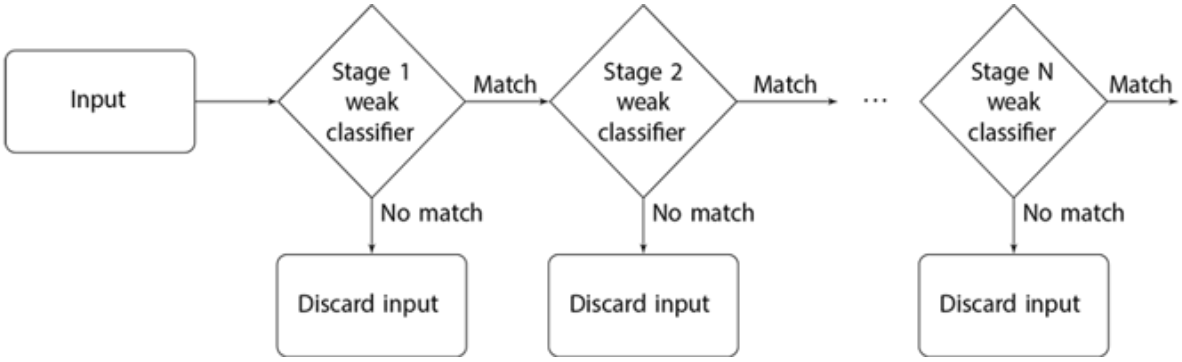


Fig: Workflow

Code:

app.py will be a simple and short collection of instruction to retrieve trained classification model and re-apply to input video frames.

```
import cv2
cars_cascade = cv2.CascadeClassifier('haarcascade_car.xml')

def detect_cars(frame):
    cars = cars_cascade.detectMultiScale(frame, 1.15, 4)
    for (x, y, w, h) in cars:
        cv2.rectangle(frame, (x, y), (x+w,y+h), color=(0, 255, 0),
thickness=2)
    return frame


def Simulator():
    CarVideo = cv2.VideoCapture('kkk.mp4')
    while CarVideo.isOpened():
        ret, frame = CarVideo.read()
        controlkey = cv2.waitKey(1)
        if ret:
            cars_frame = detect_cars(frame)
            cv2.imshow('frame', cars_frame)
        else:
            break
        if controlkey == ord('q'):
            break
    CarVideo.release()
```

```
        cv2.destroyAllWindows()


if __name__ == '__main__':
    Simulator()
```

haarcascade_car.xml is the HAAR cascade classification car file which is pre-trained to detect cars in a moving traffic

Result obtained: The image below is a still frame from the output video feed after the program was run:

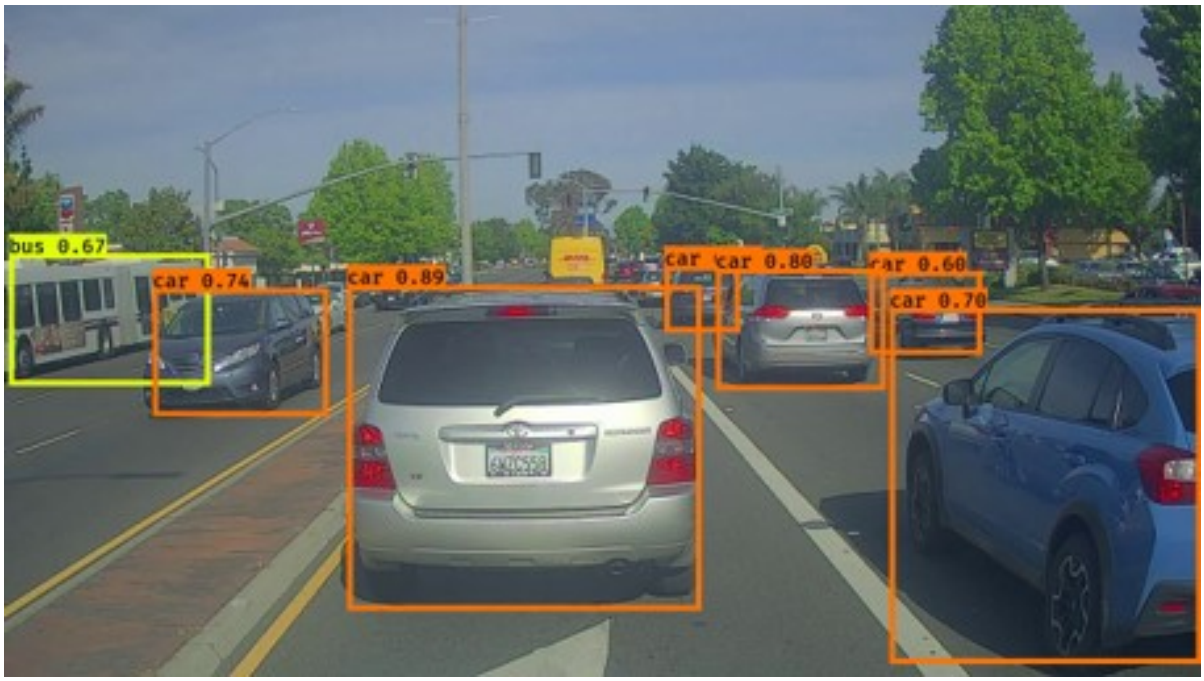


Fig: Vehicle detection from rear

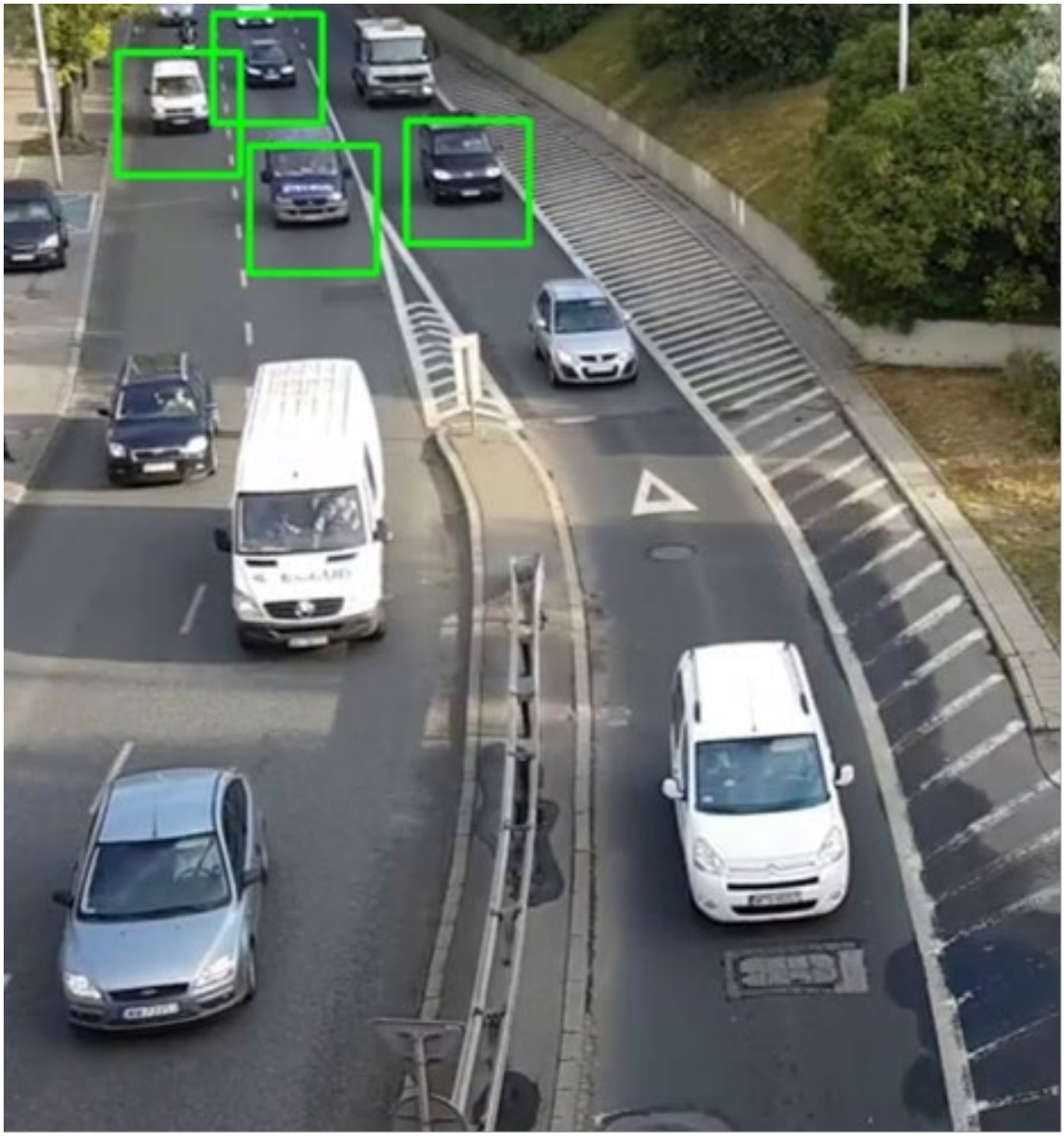


Fig: Detecting moving vehicles

The image shows capturing of vehicles into the frame above certain limit as is set into the program, so it is safe to assume that part is working fine. We can readjust that limit depending on usage and camera position. The frames are in close contact to the moving vehicles, generally in case of fast moving vehicles and this can be improved by taking less time interval in between processing frames. Overall the output looks good for vehicle detection in a moving frame and can be replicated for a raw video feed obtained from a car mounted camera.

Learnings and Conclusion:

1. Using OpenCV to isolate pixels and perform various image and graphics related operations.
2. Using Computer Vision ML techniques and learning the use various classification techniques utilized in Computer Vision.
3. Working with Cascade Classifier for object detection
4. Learning about reducing time and improving accuracy by using pre-trained models in supervised learning.

Overall, we were able to use, understand and modify the above piece of code and implement it on raw video feed and easily create our own vehicle detection pipeline.

References

- https://www.codespeedy.com/vehicle-detection-in-python-using-opencv/
- https://pyimagesearch.com/2019/12/02/opencv-vehicle-detection-tracking-and-speed-estimation/
- https://dev.to/kalebu/how-to-perform-real-time-vehicle-detection-in-python-4i9h
- https://techvidvan.com/tutorials/opencv-vehicle-detection-classification-counting/
- https://www.analyticsvidhya.com/blog/2020/04/vehicle-detection-opencv-python/
- https://dev.to/kalebu/how-to-perform-real-time-vehicle-detection-in-python-4i9h